배지혁

신입 개발자 · 2026.02 졸업

qowlgur121@gmail.com / github.com/qowlgur121 / 포트폴리오 qowlgur121.github.io

AI를 실제 업무에 붙여,
사고 없이 돌아가는 시스템으로 만듭니다.

대표 경험

주문 처리 서버 · AI 운영

개인 · 2026.09 ~ 10

Python · SQLite · Oracle Cloud

외부 API 장애에도 중복 없는 주문 처리를 설계 — 멱등 처리 · 원자적 할당 · 트랜잭션. 운영 감시는 AI 에이전트에, 결제 · 발송 권한은 코드와 사람에게

하루 최대 1,378건 실거래 처리

공공기관 AI 상담 챗봇

3인 팀장 · 2025.09 ~ 12

FastAPI · LangChain · pgvector

재학습이 필요하던 파인튜닝 챗봇을 매일 갱신되는 RAG로 재구축. LLM 의도 분류 · 조건 필터 · 규칙 재정렬로 신청 가능한 사업부터 안내

사업 1,809건 · 시설 18,318곳 · 캡스톤 우수상 · 기관 표창

OpenContext

3인 팀 · 2025.08 ~ 12

Spring Boot · Elasticsearch · MCP

사내 문서를 외부로 보내지 않는 MCP 검색 서버. BM25 + 벡터 하이브리드 검색 엔진과 2단계 조회 API(목록 → 고른 원문만) 설계 · 구현

학부 작품경진대회 은상

JH Agent OS

개인 · 2026.09 ~ 매일 사용

Python · Git · 자동 검사

여러 AI 에이전트가 같은 데이터를 동시에 쓰는 운영 계층. 현재 사실은 한 곳에만, 쓰기는 버전 확인을 거친 전이 명령으로만, AI 추론은 사실과 분리

에이전트를 바꿔도 같은 원본 · 같은 규칙 · 백업 복구 검증



학력	전북대 컴퓨터공학부 · 2026.02 졸업 **전공 3.83 · 전체 3.71** / 4.5
자격	**정보처리기사 · SQLD · ADsP**
수상	작품경진대회 은상 (2025.12) 캡스톤디자인 우수상 (2025.12) 사회서비스원 성과보고회 표창 (2025.11)
현장실습	(주)텔로스 · 백엔드 개발 · 2025.07 사내 근태 · 좌석 관리 시스템 백엔드
기술	Python · Java / Spring Boot · FastAPI PostgreSQL · SQLite · Docker · Linux

01

주문 처리 서버 · AI 운영

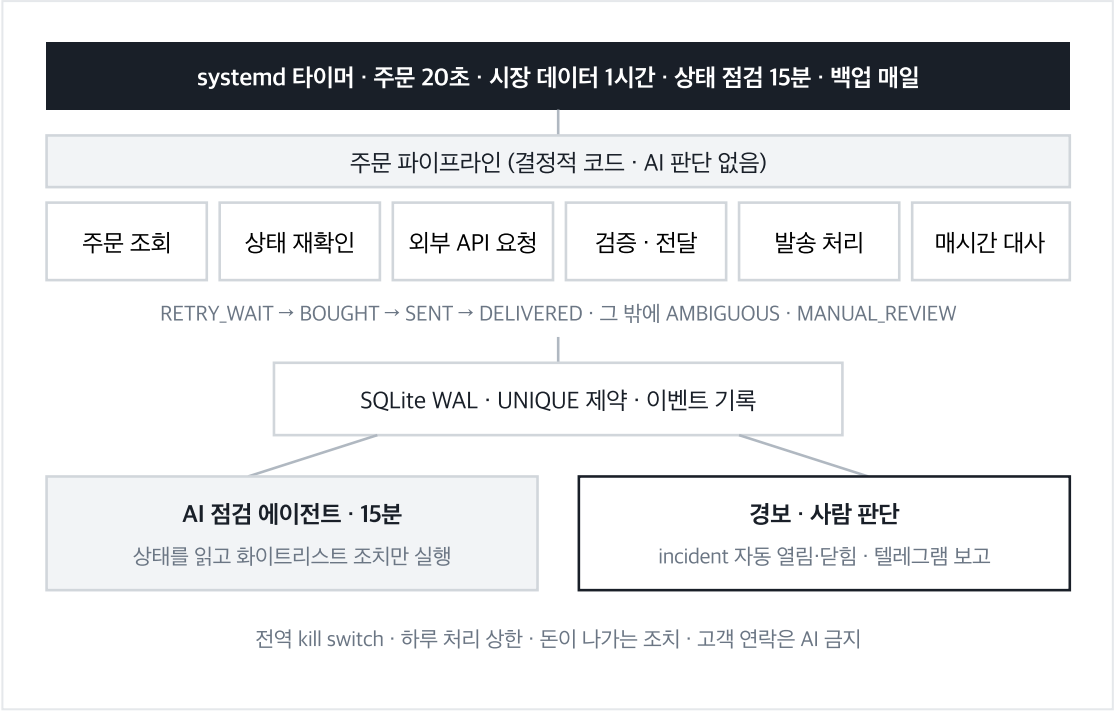
대표 프로젝트 · 개요와 구조

개인 프로젝트 · 기획부터 운영까지 직접

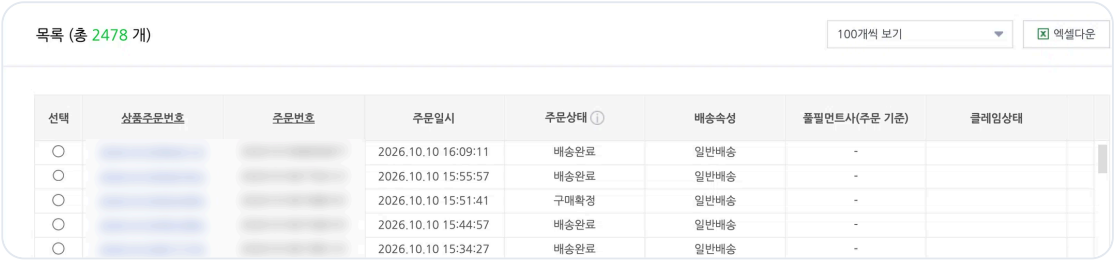
2026.09 ~ 2026.10

결제 확인 → 외부 발주 → 상품 코드 전달 → 정산 대조까지 사람 개입 없이 이어지는 주문 파이프라인입니다. 예외만 사람에게 올라옵니다.

Stack	Python · SQLite(WAL) · unittest
Infra	Oracle Cloud · Ubuntu · systemd 타이머 · 매일 백업
연동	네이버 커머스 API · 외부 주문 API · 문자 발송 · Telegram 알림
AI	AI 점검 에이전트(15분) · 허용된 조치만 · 결제 · 발송 권한 없음



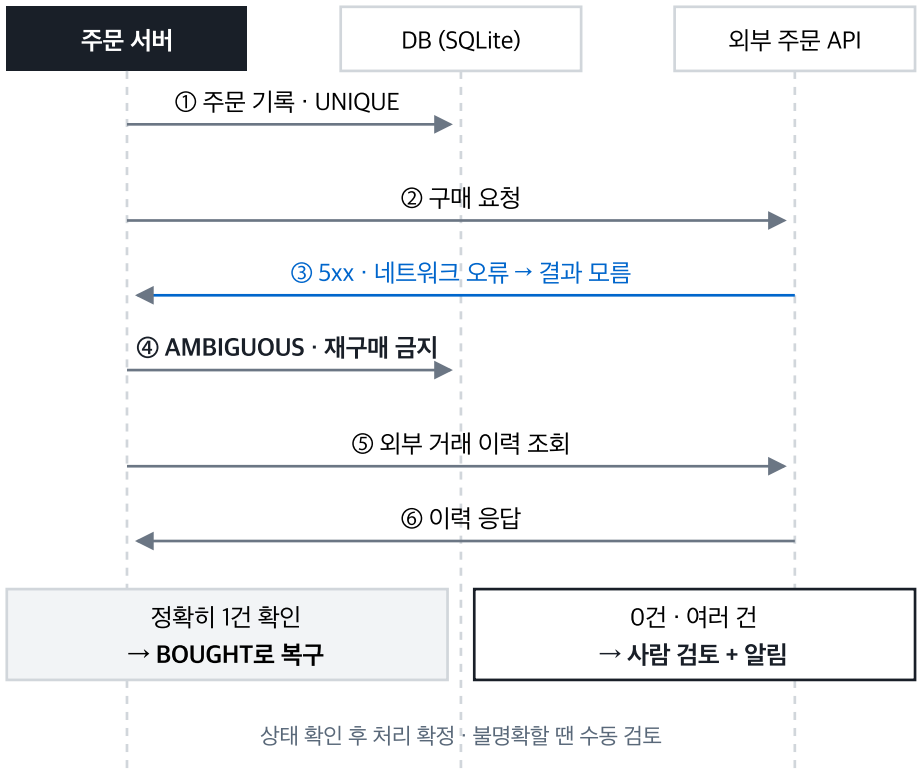
시스템 구조. 회색 상자는 AI 에이전트 · 테두리 상자는 사람 판단



판매자센터 주문 목록 (주문번호는 가림)

외부 API 응답이 끊겨도 같은 주문이 두 번 처리되지 않게 했습니다

문제 해결 1



구매 결과를 모를 때의 처리 순서

문제

외부 주문 API의 응답이 끊기면 처리됐는지 알 수 없었고, 그냥 재시도하면 같은 주문이 두 번 처리될 수 있었습니다.

설계 판단

자동 재시도	동일 요청이 중복 실행될 수 있음
즉시 실패	실제 처리 여부와 내부 상태가 어긋날 수 있음
선택한 방식	**상태 보류 → 거래 이력 확인 → 결과 확정**

결과가 불명확하면 **AMBIGUOUS**로 보류하고, 외부 거래 이력에서 일치하는 내역을 찾아 복구했습니다. 애매한 건은 사람 확인으로 넘겼습니다.

결과

불명확한 주문을 재실행하지 않고 **대조 후 확정**하는 순서로 판매 6일째 하루 1,378건을 처리했습니다. 발송 직전에도 같은 1회용 상품 코드가 다른 주문에 나간 적이 있는지 한 번 더 확인하고, 겹치면 보내지 않고 멈추게 했습니다.

서버 1대(Oracle Cloud ARM 1 OCPU · 6GB) · SQLite WAL · 20초 주기 조회
핵심 로직 예제 · github.com/qowlgur121/order-fulfillment-guard

같은 코드를 두 처리가 동시에 잡아도 한쪽만 성공하게 했습니다

문제 해결 2

① 같은 주문이 두 번 들어와도 한 건만

```
UNIQUE INDEX orders(channel, channel_order_id)
```

20초마다 조회해 같은 주문을 또 읽어도 새로 생기지 않음

② 같은 상품 코드는 한 번만 등록

```
artifacts.fingerprint UNIQUE
```

공급처가 같은 코드를 두 번 돌려줘도 두 번째는 거부

③ 한 코드는 한 주문에만 (조건부 UPDATE)

```
UPDATE artifacts SET state='RESERVED',
reserved_order_id=?
WHERE id=? AND state='VALIDATED'
AND reserved_order_id IS NULL;
```

바뀐 행이 1개가 아니면 예외 → 트랜잭션 전체 되돌림

상태 변화는 events 표에 모두 기록

어느 주문에 어떤 코드가 언제 묶였는지 추적 가능

중복을 DB가 마지막에 막는 구조

문제

주문 조회는 20초마다, 점검은 15분마다 따로 돌았습니다. 같은 주문을 두 번 읽거나, 두 처리가 같은 1회용 코드를 동시에 잡으면 **한 코드가 두 고객에게 가거나 같은 주문에 코드가 두 번** 나갈 수 있었습니다.

설계 판단

코드에서 먼저 조회 후 저장	조회와 저장 사이에 다른 처리가 끼어들 수 있음
별도 잠금 서버 · 큐	서버 1대 규모에 운영할 것만 늘어남
선택한 방식	**DB 제약 + 조건부 UPDATE로 DB가 마지막에 막게**

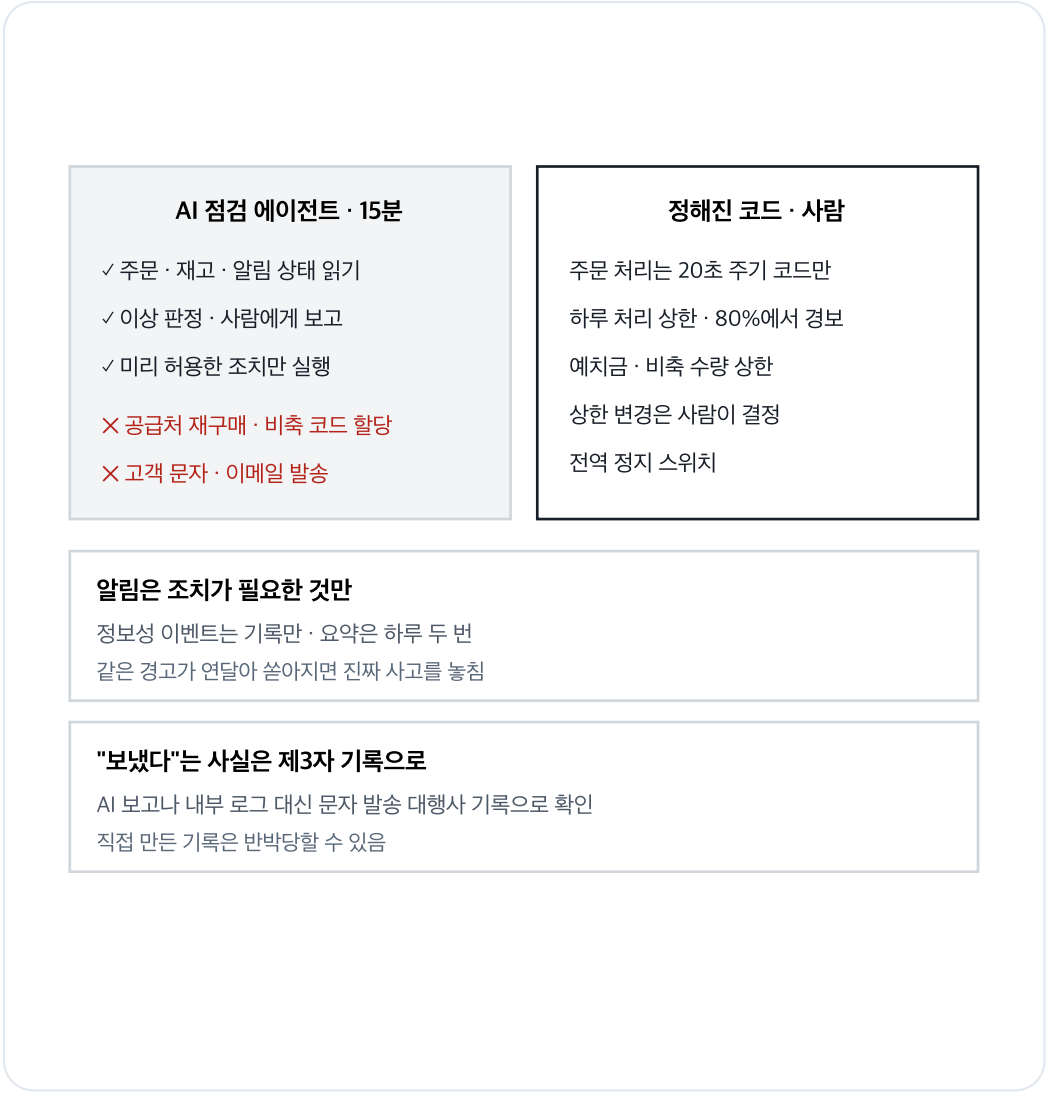
SQLite를 WAL 모드로 두고, 코드 할당과 주문 갱신을 **한 트랜잭션**으로 묶었습니다. 발송 기록 저장이 실패하면 코드 할당까지 함께 되돌립니다.

결과

두 처리가 같은 코드를 동시에 잡아도 **한쪽만 성공**하고 나머지는 예외로 멈춥니다. 중복 코드 거부 · 원자 할당 · 실패 시 되돌림을 각각 테스트로 고정했습니다.

AI 에이전트에게 감시는 맡기고, 돈이 나가는 권한은 주지 않았습니다

문제 해결 3



AI 에이전트 · 코드 · 사람의 역할 나누기

문제

혼자 24시간 지켜볼 수 없어 AI 점검 에이전트를 붙였습니다. 하지만 1회용 코드와 선결제가 걸린 일이라, 에이전트가 잘못 판단하면 **되돌릴 수 없는 손해**가 납니다.

설계 판단

에이전트가 처리까지	같은 상황에서도 매번 다르게 판단할 수 있음
사람이 모든 주문 확인	하루 1,000건 넘게 오면 감당 불가
선택한 방식	**처리는 코드, 감시는 에이전트, 돈 · 한도는 사람**

에이전트가 해서는 안 되는 일(재구매 · 코드 할당 · 고객 발송)을 목록으로 막고, 규칙으로 처리할 수 있는 일에는 AI를 부르지 않았습니다.

결과

에이전트는 상태를 읽고 보고만 하고, **돈이 나가는 조치는 모두 정해진 코드 규칙과 제결정으로만** 실행됩니다. 처리 상한은 정상 판매가 확인될 때만 늘어나게 했습니다.

02

공공기관 AI 상담 챗봇 · RAG

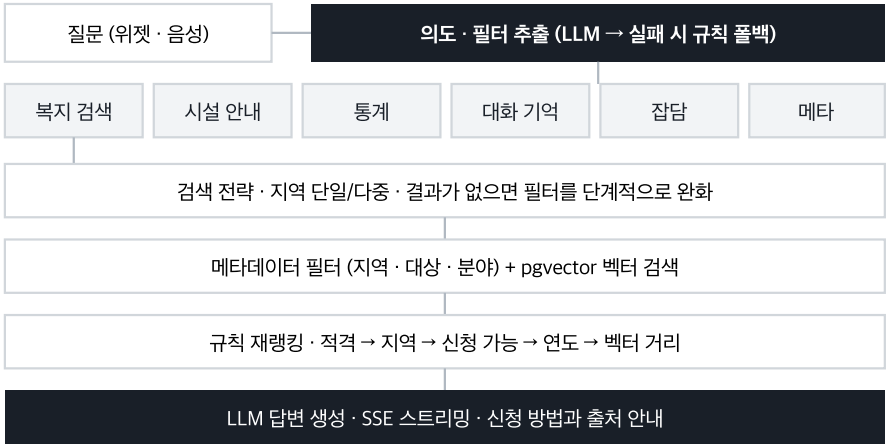
대표 프로젝트 · 개요와 구조

산학 프로젝트 · 3인 팀 팀장

2025.09.22 ~ 2025.12.05

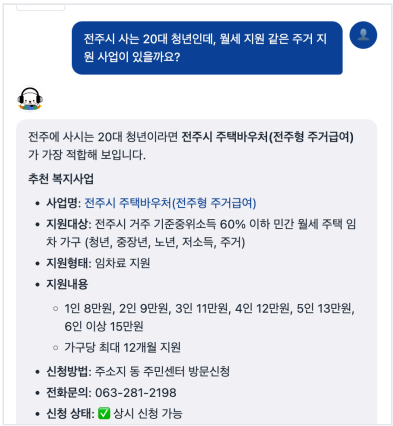
공공기관 홈페이지에 붙는 AI 상담 챗봇을 매일 갱신되는 RAG 구조로 새로 만들었습니다. 3인 팀장으로 기관과 요구사항을 맞추고 서버 · 검색을 맡았습니다.

협력	전북특별자치도사회서비스원
Backend	FastAPI · SSE 스트리밍 · APScheduler · Docker Compose
AI · 검색	LangChain · bge-m3 임베딩 · PostgreSQL + pgvector
역할	**내 담당** 요구사항 협의 · 설계 · 서버 · 검색 **팀원** 자료 조사 · 발표 · 문서



데이터: 게시판 · 복지지도 크롤링 → 정제 → bge-m3 임베딩 → pgvector (매일 03:00 증분)

질문 하나가 답이 되기까지의 흐름





조건을 말하면 받을 수 있는 사업과 신청 방법 안내 / 증상을 말하면 가까운 의료기관을 거리순 안내

재학습 없이 매일 새 사업이 반영되는 RAG 구조로 바꿨습니다

문제 해결 4



기존 방식과 새 방식 비교

문제

기존 챗봇은 파인튜닝 기반이라 학습한 Q&A 안에서만 답했고, 새 사업을 넣으려면 다시 학습해야 해서 신청 기간이 지난 사업을 안내하기도 했습니다.

설계 판단 · 기관 조건에서 출발

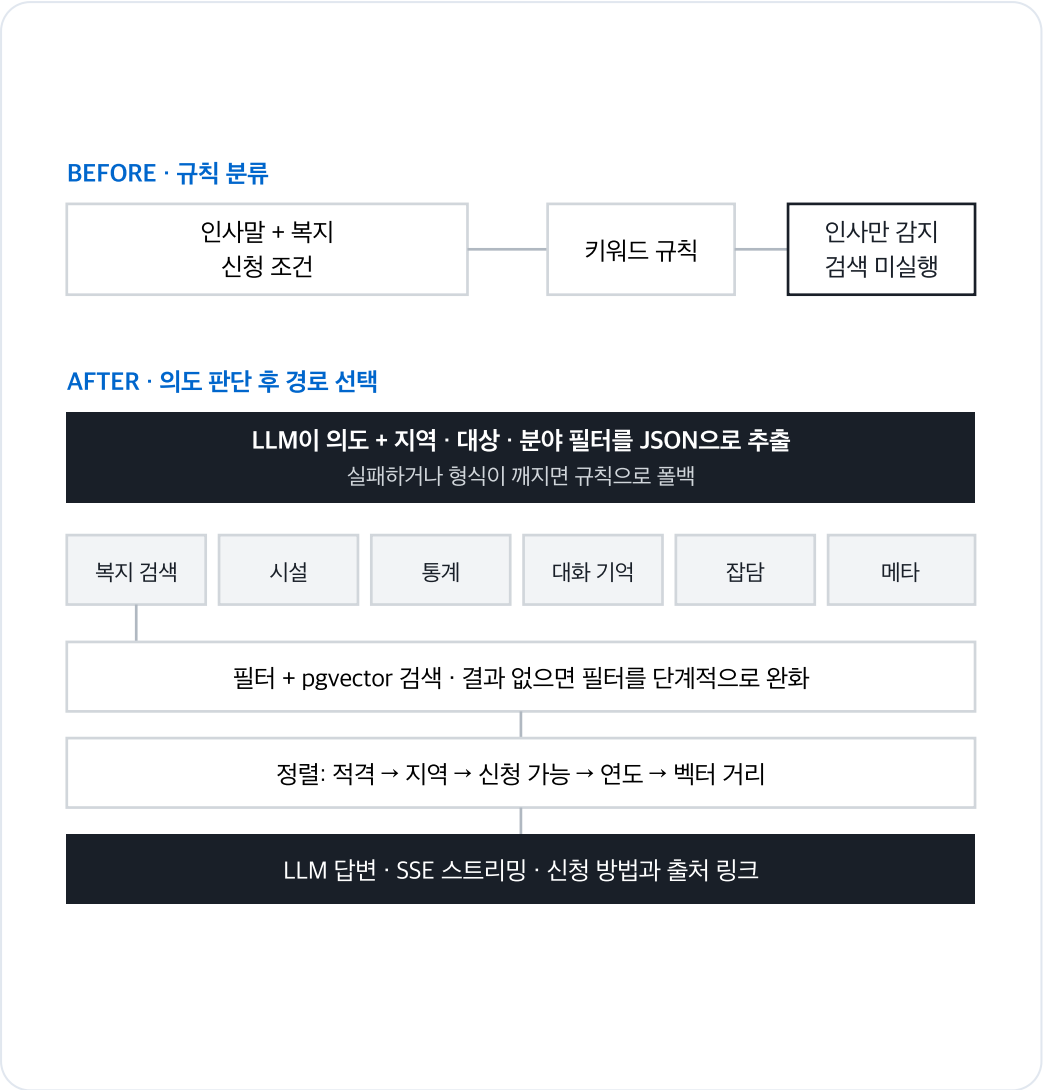
데이터 확보	연동 API · DB 계정 요청이 비용 문제로 거절됨 → 공개 게시판 11종을 직접 수집, 바뀐 사업만 매일 03:00 반영
서버 언어	Spring은 공공기관 전자정부 프레임워크 부담 → 수집 · 검색 · LLM을 Python(FastAPI) 하나로
벡터 저장소	운영 담당자가 개발자가 아님 → 전용 벡터 DB 대신 PostgreSQL + pgvector로 조건 필터와 의미 검색을 한 쿼리에
기존 사이트	**원본 수정 권한 없음 → 스크립트 한 줄로 붙는 위젯(Custom Element)**

결과

재학습 없이 복지사업 1,809건이 매일 새벽 반영되고, 찾은 원문과 공식 게시글 링크로 답합니다. 키워드 검색으로 0건이던 "치매" 질문에서 관련 사업 20건을 찾았습니다.

LLM이 의도와 자격 조건을 뽑고, 신청 가능한 사업부터 답하게 했습니다

문제 해결 5



질문 처리 흐름 개선 전후

문제

"50대인데 장애가 있고 전주 살아요"처럼 조건이 섞인 질문은 키워드 규칙으로 처리할 수 없었고, 검색해도 기간이 지났거나 지역이 다른 사업이 먼저 나왔습니다.

설계 판단

규칙만 추가	표현이 달라질 때마다 예외 유지보수 필요
LLM 단독	응답 실패·형식 오류에 대한 별도 처리 필요
선택한 방식	**LLM으로 의도 추출 + 검증 후 규칙 폴백**

LLM이 의도 · 지역 · 대상 · 분야를 JSON으로 뽑아 6개 경로로 보내고, 형식이 깨지면 규칙 분류로 넘어갑니다. 결과는 **자격 → 지역 → 신청 가능 → 연도 → 벡터 거리** 순으로 다시 정렬했습니다.

결과

조건이 섞인 질문도 복지 검색으로 이어지고, LLM이 실패해도 규칙 경로로 답합니다.

OpenContext · 사내 문서 AI 검색 MCP 서버

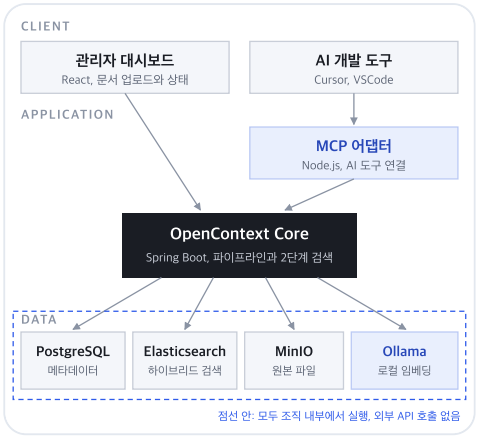
개요와 구조

3인 팀 · 오픈소스 개발자대회

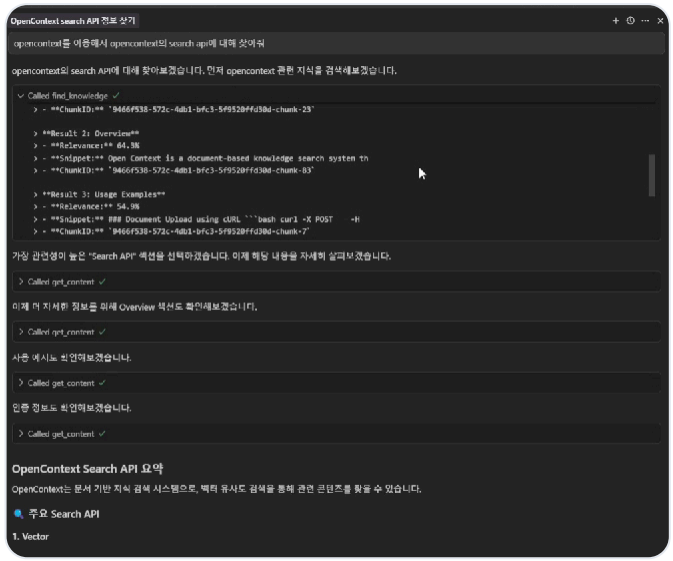
2025.08 ~ 2025.12 · github.com/OpenContextAI/open-context

AI 코딩 도구가 사내 문서를 검색하게 해 주는 자체 설치형 MCP 검색 서버입니다. 문서 처리와 임베딩이 모두 회사 안에서 끝납니다.

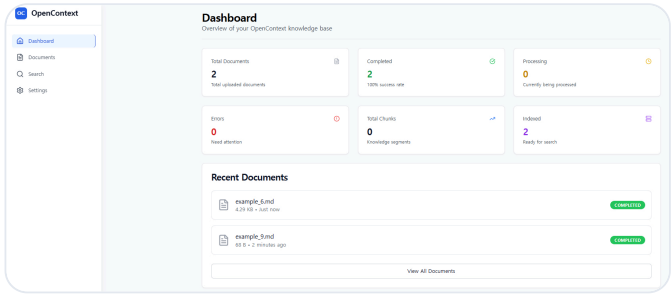
Backend	Java 21 · Spring Boot 3.3 · PostgreSQL
검색	Elasticsearch(BM25) · Ollama bge-m3 로컬 임베딩
연동	MCP · Cursor · VS Code
역할	**내 담당** 하이브리드 검색 엔진 · 2단계 조회 API · DB · 배포 **팀원** 문서 수집 파이프라인 · MCP 어댑터



전체 구조. 점선 안은 모두 조직 내부에서 실행



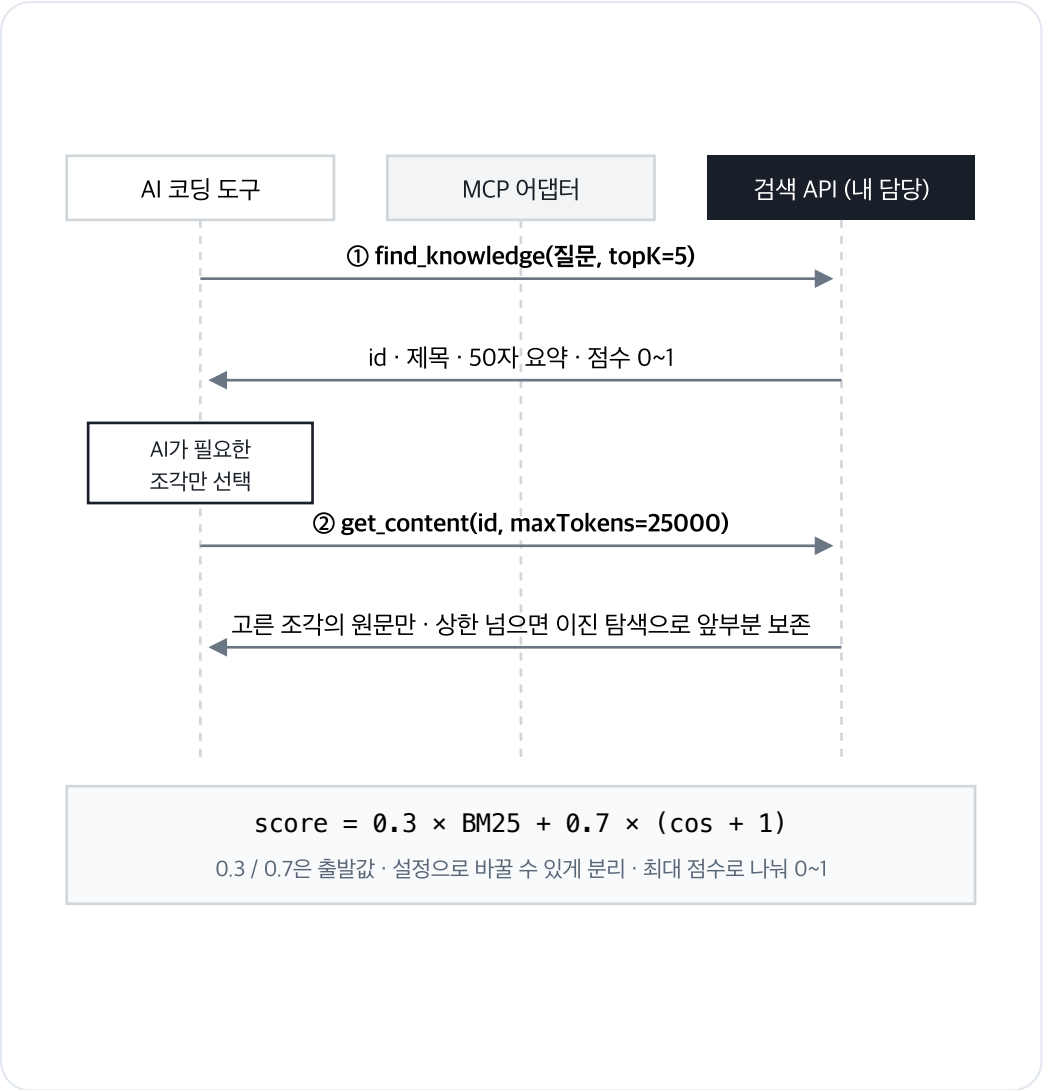
AI 도구가 find_knowledge로 고르고 get_content로 원문만 가져옴



관리자 화면 · 문서 처리 상태

AI가 목록을 먼저 보고 필요한 원문만 가져가는 2단계 검색 API

문제 해결 6



2단계 검색 호출 순서와 점수 계산

문제

검색한 원문을 한 번에 다 넘기면 컨텍스트가 커지고 AI가 쓸 자료를 고르기 어려웠습니다.

해결

find_knowledge에서 제목·요약·관련도 목록을 먼저 반환하고, AI가 선택한 문서만 **get_content**로 원문을 가져가도록 나눴습니다.

검색은 BM25 키워드 점수와 벡터 유사도를 합치고, 순위와 함께 관련도 점수를 돌려줬습니다.

결과

AI 도구가 **먼저 고르고 필요한 원문만 읽는 검색 경로**를 만들었습니다. 문서 처리와 임베딩은 모두 조직 안에서 돕니다.

04

JH Agent OS · 여러 AI 에이전트가 같은 데이터를 함께 쓰게 하는 운영 계층

개요와 구조

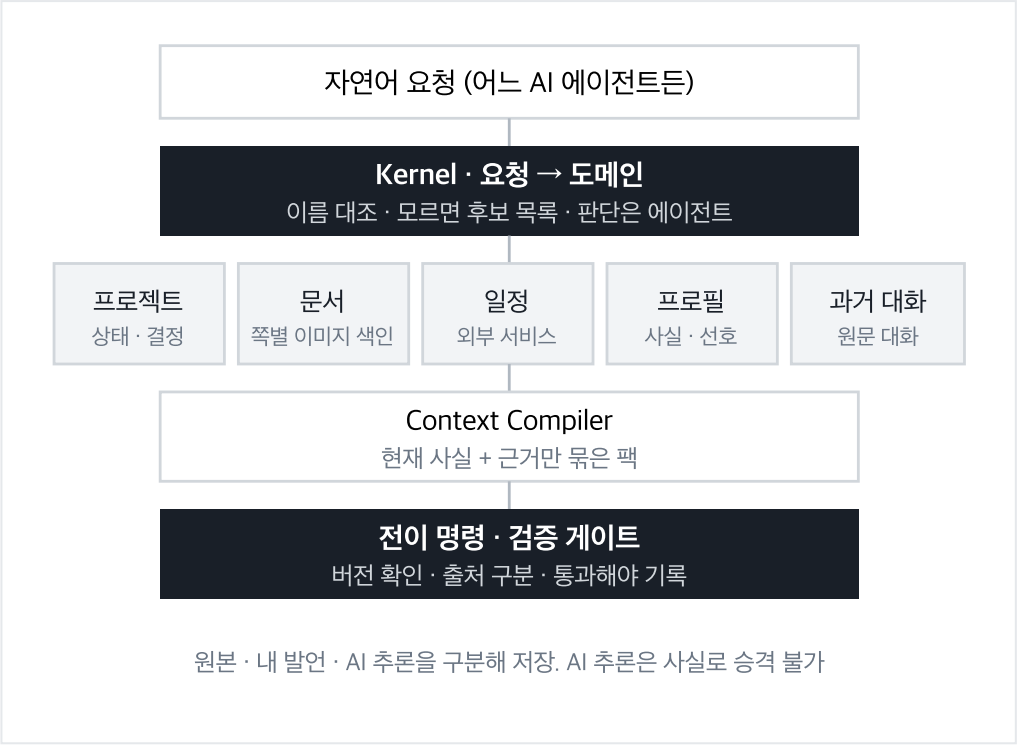
개인 · 직접 설계 · 매일 사용

2026.09 ~ 현재

Claude Code · Codex 등 여러 AI 에이전트가 제 일정 · 공부 · 지원 기록을 동시에 읽고 고칩니다. 에이전트가 무엇을 사실로 믿고, 어디까지 쓸 수 있는지를 코드로 정한 운영 계층입니다.

정본	도메인마다 원본 · 현재 상태 · 결정 기록을 나누고, 현재 사실은 한 곳에만
요청 해석	시스템은 도메인 목록과 현재 상태만 제공 · 의미 판단은 대화 맥락을 아는 에이전트가
컨텍스트	요청마다 필요한 사실과 근거만 묶어, 어느 에이전트에든 같은 형식으로
쓰기 통제	허용된 전이 명령으로만 기록 · 이전 버전이 바뀌었으면 거부
출처	원본 · 사용자 발언 · AI 추론을 구분해 저장 · 추론은 사실이 되지 못함
규칙 주입	모든 에이전트의 시작 파일에 최소 계약을 넣어, 문서를 안 읽어도 지키게
Stack	Python · Git · JSON · 자동 검사

에이전트를 바꿔도 같은 원본과 같은 규칙을 보도록, **도구가 아니라 데이터와 권한**을 설계했습니다.



요청이 들어와서 기록되기까지의 흐름

동시 작업 · 복구

작업 보존	여러 세션의 끝나지 않은 작업을 커밋과 분리해 시점별 스냅샷으로 보존 (첫 실행 105개, 그중 12개는 백업 없던 파일)
복구 검증	이력 전체를 파일 하나로 백업하고 실제 복구 시험 · 일반 clone이 스냅샷을 빠뜨리는 것을 확인해 복구 절차를 바꿈